

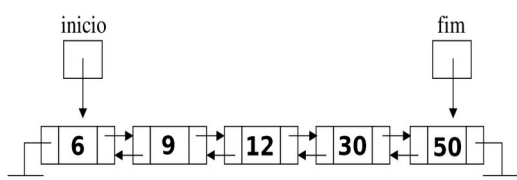
1) Em relação às frases abaixo, explique rapidamente as afirmações ou discorde, justificando.

- A fila do RU é um patrimônio imemorial da UFSC, mas ela não é uma fila FIFO. (0,5 pt)
- Uma pilha de pratos não é uma pilha LIFO. (0,5 pt)
- A implementação de esquema de fila circular em Array é útil para evitar movimentação de dados (0,5).

2) Sabemos que listas encadeadas são estruturas intrinsecamente sequenciais - e isso gera um monte de limitações sobre o desempenho de operações importantes, como a busca.

a) Se não tivermos a informação de quantos elementos há em uma lista, como podemos descobrir e acessar o elemento central de uma lista de 1001 elementos? Descreva (com suas palavras) o algoritmo para tal e diga qual o desempenho geral do mesmo (quantos passos precisamos dar para encontrar e acessar o elemento central) (1,0 pt)

b) Se tivermos uma lista duplamente encadeada (como a do desenho abaixo), podemos melhorar o tempo de resposta para essa pergunta. Para isso, podemos ter 2 iteradores, um que parte do início em direção ao final e outro que parte do final em direção ao início e; alternativamente, damos passos em cada um dos iteradores. Quando ambos estiverem no mesmo elemento, este será o elemento central, certo? Quantos passos o algoritmo terá dado para chegar aí? Implemente o código deste procedimento em uma lista duplamente encadeada (use os nomes das classes, funções e variáveis, conforme exemplo abaixo) e que, ao final, informe qual é o elemento central e também quantos elementos há na lista. (2,5 pts)



```

class No:
    def __init__(self, d):
        self.dado = d
        self.prox = None
        self.ant = None

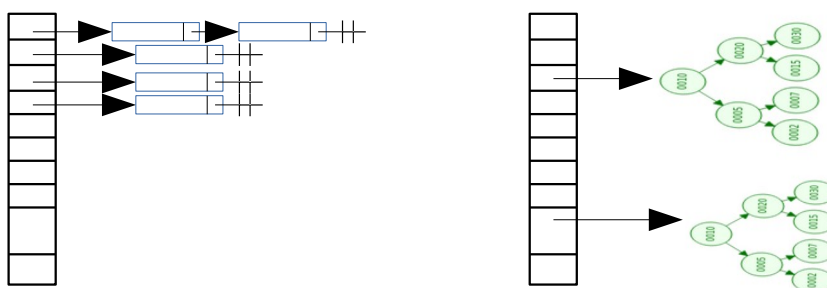
class lista_2mte:
    def __init__(self):
        self.inicio = None
        self.fim = None
        
```

```

# exemplo de fç de
# inserção na frente
def insereFrente(self, novoDd):
    novoNo = No(novoDd)
    novoNo.prox = self.inicio
    if self.inicio is not None:
        self.inicio.ant = novoNo
    self.inicio = novoNo
        
```

....

3) Um hash tradicional é composto por uma tabela de espalhamento em um array de listas (ou seja, o chamado de Encadeamento direto, figura da esquerda). Imagine que, em vez de usar um array de listas, alguém resolve implementar como um array de árvores binárias de busca (figura da direita). Faz sentido? Analise os aspectos que julgar importante. (2,5 pts.)



4) Comente, justifique, responda (0,5 pt cada item)

- Retirada de um elemento de uma árvore binária de busca: para a retirada de um nó que tenha 2 filhos, qual é; ou melhor, quais são os procedimentos básicos para realizar essa tarefa? Por que essas são as formas recomendadas?
- Quais são e para quê servem os algoritmos clássicos de caminhadas em árvores binárias?
- A busca em uma árvore binária é geralmente melhor que em uma lista ordenada? Por quê?
- Uma árvore binária degenerada apresenta desempenho de busca pior que uma lista.
- O que define a distribuição dos elementos e a organização da árvore binária de busca (não AVL) são os algoritmos de inclusão e de exclusão de elementos? Como assim?